

Praktikum 6 - Gruppe 10

None

None

None

Table of contents

1. Praktikum 6 - Gruppe 10	3
1.1 Teilaufgabe 1: Datenvorbereitung	3
1.2 Teilaufgabe 2: One-Hot-Encoding und Apriori-Algorithmus	3
1.3 Teilaufgabe 3: Generierung und Analyse von Assoziationsregeln	4
1.4 Teilaufgabe 4: Interpretation und geschäftliche Anwendung	5

1. Praktikum 6 - Gruppe 10

1.1 Teilaufgabe 1: Datenvorbereitung

1.1.1 3.

```
SELECT
  ss.ss_ticket_number AS L_ORDERKEY,
  i.i_category        AS L_CATEGORY
FROM SNOWFLAKE_SAMPLE_DATA.TPCDS_SF10TCL.STORE_SALES ss
JOIN SNOWFLAKE_SAMPLE_DATA.TPCDS_SF10TCL.ITEM i
  ON ss.ss_item_sk = i.i_item_sk
WHERE ss.ss_quantity > 0
      AND i.i_category IS NOT NULL
LIMIT 100000
```

1.1.2 4. Convert to dataframe:

```
df = session.sql("""
SELECT
  ss.ss_ticket_number AS L_ORDERKEY,
  i.i_category        AS L_CATEGORY
FROM SNOWFLAKE_SAMPLE_DATA.TPCDS_SF10TCL.STORE_SALES ss
JOIN SNOWFLAKE_SAMPLE_DATA.TPCDS_SF10TCL.ITEM i
  ON ss.ss_item_sk = i.i_item_sk
WHERE ss.ss_quantity > 0
      AND i.i_category IS NOT NULL
LIMIT 100000
""").to_pandas()

transactions = (df.groupby("L_ORDERKEY")["L_CATEGORY"]
                .apply(lambda s: list(pd.unique(s)))
                .tolist())
```

1.1.3 5. Menge der Transaktionen, eindeutige Categorien und erste 5 Transaktionen

```
transactions = (df.groupby("L_ORDERKEY")["L_CATEGORY"]
                .apply(lambda s: list(pd.unique(s))) # removes duplicates
                .tolist()) # list-of-lists

# Required checks
print("Transactions:", len(transactions))
print("Unique categories:", df["L_CATEGORY"].nunique())
print("First 5 transactions:", transactions[:5])
```

1.2 Teilaufgabe 2: One-Hot-Encoding und Apriori-Algorithmus

```
te = TransactionEncoder()
te_arr = te.fit(transactions).transform(transactions)

onehot = pd.DataFrame(te_arr, columns=te.columns_)

onehot.shape # (number_of_transactions, number_of_categories)

freq_itemsets = apriori(onehot, min_support=0.20, use_colnames=True)
freq_itemsets = freq_itemsets.sort_values("support", ascending=False)
```

```
freq_itemsets.head(20)
freq_itemsets["itemset_size"] = freq_itemsets["itemsets"].apply(len)
freq_itemsets["itemset_size"].value_counts().sort_index()
```

Result:

```
(10509, 10)
```

itemset_size count

```
1      10
2      45
3     120
```

1.3 Teilaufgabe 3: Generierung und Analyse von Assoziationsregeln

1.3.1 1:

```
rules = association_rules(freq_itemsets, metric="confidence", min_threshold=0.1)
rules_sorted_conf = rules.sort_values("confidence", ascending=False)

len(rules_sorted_conf), rules_sorted_conf.head(10)[
    ["antecedents", "consequents", "support", "confidence", "lift"]
]
```

Result:

	antecedents	consequents	support	confidence	lift
(810,					
217	(Children, Men)	(Books)	0.242269	0.650984	1.078033
211	(Shoes, Men)	(Books)	0.242364	0.650575	1.077354
247	(Children, Men)	(Women)	0.242078	0.650473	1.073296
91	(Jewelry, Sports)	(Music)	0.246741	0.650038	1.075279
325	(Children, Shoes)	(Books)	0.241317	0.649923	1.076275
212	(Books, Men)	(Shoes)	0.242364	0.649745	1.090936
218	(Books, Men)	(Children)	0.242269	0.649490	1.078447
342	(Shoes, Jewelry)	(Books)	0.240936	0.649397	1.075404
290	(Home, Men)	(Women)	0.241698	0.648787	1.070513
314	(Books, Men)	(Women)	0.241507	0.647449	1.068306)

1.3.2 2:

stats (min, max, mean)

```
stats = rules[["support", "confidence", "lift"]].agg(["min", "max", "mean"])
stats
```

Result:

	support	confidence	lift
min	0.233	0.3866	1.0337
max	0.383	0.651	1.0909
mean	0.255	0.5296	1.0563

top 10 Regeln nach lift

```
top10_lift = rules.sort_values("lift", ascending=False).head(10)[
    ["antecedents", "consequents", "support", "confidence", "lift"]
]
```

```
]
top10_lift
```

top 10 Regeln nach confidence

1.3.3 3:

welche Regeln hat Lift > 1.0

```
rules_lift_gt_1 = rules[rules["lift"] > 1.0] \
    .sort_values(["lift", "confidence"], ascending=False)[
        ["antecedents", "consequents", "support", "confidence", "lift"]
    ]

len(rules_lift_gt_1), rules_lift_gt_1.head(20)
```

Lift > 1.0 means the consequent happens more often together with the antecedent than it happens in general

welche Regeln hat Lift > 1.0 UND confidence > 0.5

```
rules_lift_gt_1_conf_gt_05 = rules[(rules["lift"] > 1.0) & (rules["confidence"] > 0.5)] \
    .sort_values(["lift", "confidence"], ascending=False)[
        ["antecedents", "consequents", "support", "confidence", "lift"]
    ]

len(rules_lift_gt_1_conf_gt_05), rules_lift_gt_1_conf_gt_05.head(20)
```

Natural language sentences:

```
def rule_to_text(row):
    X = ", ".join(list(row["antecedents"]))
    Y = ", ".join(list(row["consequents"]))
    return (f"Wenn Kunden {X} kaufen, dann kaufen sie auch {Y} in "
            f"{row['confidence']*100:.1f}% der Fälle "
            f"(Support {row['support']*100:.1f}%, Lift {row['lift']:.2f}).")

texts = rules_lift_gt_1_conf_gt_05.head(5).apply(rule_to_text, axis=1).tolist()
texts
```

```
[
0:"Wenn Kunden Men, Books kaufen, dann kaufen sie auch Shoes in 65.0% der Fälle (Support 24.2%, Lift 1.09)."
```

```
1:"Wenn Kunden Books, Shoes kaufen, dann kaufen sie auch Men in 64.0% der Fälle (Support 24.2%, Lift 1.08)."
```

```
2:"Wenn Kunden Music, Sports kaufen, dann kaufen sie auch Jewelry in 64.6% der Fälle (Support 24.7%, Lift 1.08)."
```

```
3:"Wenn Kunden Men, Books kaufen, dann kaufen sie auch Children in 64.9% der Fälle (Support 24.2%, Lift 1.08)."
```

```
4:"Wenn Kunden Men, Children kaufen, dann kaufen sie auch Books in 65.1% der Fälle (Support 24.2%, Lift 1.08)."
```

```
]

```

1.4 Teilaufgabe 4: Interpretation und geschäftliche Anwendung

1.4.1 1.

- Support = 0.15 bedeutet: In deinem Datensatz enthalten 15% aller Transaktionen gleichzeitig den Antecedent und den Consequent der Regel
- Confidence = 0.8 bedeutet: Wenn der Antecedent in einem Warenkorb vorkommt, dann kommt der Consequent in 80% dieser Fälle ebenfalls vor

- Lift = 2.5 bedeutet: Der Consequent tritt bei Vorhandensein des Antecedents 2,5-mal so häufig auf, wie man es erwarten würde, wenn beide unabhängig wären (starker positiver Zusammenhang)
- Lift = 1 bedeutet: Antecedent und Consequent sind (ungefähr) unabhängig, d.h. der Antecedent erhöht (oder senkt) die Wahrscheinlichkeit des Consequents nicht

1.4.2 2.

```
strong = rules[(rules["lift"] > 1.2) & (rules["confidence"] > 0.7)] \
    .sort_values(["lift", "confidence"], ascending=False)[
        ["antecedents", "consequents", "support", "confidence", "lift"]
    ]

strong
```

Result: empty, max lift = 1.0909 und max confidence = 0.6509844029659934

1.4.3 3.

- Regeln mit hohem Lift/Confidence zeigen, was oft zusammen gekauft wird – diese Sachen kann man im Laden/Shop näher zusammen platzieren oder zusammen bewerben
- Empfehlungssystem: Regeln mit hoher Confidence passen gut fuer "Kunden kauften auch ..." weil sie relativ zuverlässig sagen, was als Nächstes wahrscheinlich ist
- Statistisch da, aber unpraktisch: Ja, z.B. wenn der Lift nur knapp über 1 ist (fast Zufall), wenn es zu allgemeine Kategorien sind, oder wenn Produkte nicht gleichzeitig verfügbar/verkaufbar sind